오픈소스 SW 기초 3분반 프로젝트 제안서

4조 한상윤, 양진혁, 박현준, 안병규

프로젝트 주제 : 제약 조건 충족 문제(CSP) 기반 최적 시간표 자동 생성 챗봇 개발

I. 프로젝트 개요

1. 제안 배경

1.1 문제 인식

매 학기마다 수강신청 기간이 되면 학생들은 단순하게 듣고 싶은 과목을 고르는 것을 넘어서, 수십 가지 조건을 동시에 만족하는 시간표를 손으로 직접 설계해야된다. 예를 들어, 금요일 공강 시간 확보, 특정 전공 필수 과목 포함, 아침 수업 회피, 복수전공 이수 요건 충족, 선수과목 이수 여부, 건물 간 이동 시간 고려 등이 모두 하나의 시간표 내에서 동시에 해결되어야 한다.

이러한 조건들을 수작업으로 처리하기에는 경우의 수가 지나치게 많다. 예를 들어서 수강 가능한 과목이 50개이고, 그 중 6과목을 선택한다고 하면 단순 조합만 해도 $C(50,6) = \frac{50!}{6! \times 44!} = 15{,}890{,}700$ 약 1,600만 가지에 달한다. 여기에 분반 선택, 시간대 제약, 전공 이수 구분까지 더하면 실질적인 탐색 공간은 수억 가지를 넘어설 것이다. 학생들이 이를 일일이 대조하며 완벽한 시간표를 찾는 데 많은 시간을 소비하는 것은 필연적인 결과다.

1.2 기존 해결책의 한계

현재 학생들이 많이 사용하는 방법은 에브리타임 시간표 앱에서 과목을 하나씩 선택해서 배치하는 방식인데, 이는 여전히 모든 탐색을 사람이 직접 수행해야 한다는 점에서 본질적인 문제점을 해결하지는 못한다. 또한 에브리타임에는 '시간표 마법사'라는 기능이 있는데, 사용자가 과목을 그룹으로 묶어 담아두면 각 그룹에서 한 과목씩 꺼내서 가능한 모든 조합을 탐색한 뒤 만들어질 수 있는 시간표를 보여주는 방식이다. 이것은 현재 존재하는 도구 중에서 가장 자동화에 가까운 접근이다.

그러나 이 기능은 결정적인 한계를 지닌다. 탐색해야할 경우의 수가 일정 기준을 초과하면 마법사 실행 자체가 차단된다. 사용자가 선택지를 많이 추가할수록 오히려 기능을 사용할 수 없게 되는 역설적인 구조다. 이는 단순 Brute-Force 알고리즘 방식의 근본적인 한계로, 탐색 공간이 커질수록 계산 비용이 폭발적으로 증가하기 때문이다. 본 프로젝트는 이러한 한계를 CSP 알고리즘을 통해 극복해보고자 한다.

2. 제안 내용

2.1 서비스 개요

본 프로젝트는 제약 조건 충족 문제(CSP) 알고리즘과 자연어 기반 챗봇 인터페이스를 결합한 웹 서비스로, 사용자가 언하는 조건을 입력하면 조건을 만족하는 최적 시간표를 자동으로 생성해준다. 단국대학교 웹정보 시스템-강의 시간표의 강의 데이터를 기반으로 동작하며, 단일 전공생부터 복수 전공생, 부전공생까지 다양한 사용자 유형을 지원한다.

2.2 왜 CSP(Constraint Satisfaction Problem) 알고리즘인가?

CSP는 변수(Variable), 도메인(Domain), 제약 조건(Constraint)의 세 가지의 요소로 문제를 정의하고, 모든 제약을 동시에 만족하는 해를 탐색하는 알고리즘이다. 시간표 문제에 대입하면 변수는 수강할 각 과목, 도메인은 해당 과목이 개설된 모든 분반과 시간 슬롯, 제약 조건은 시간 충돌 금지, 특정 요일 공강, 선수 과목 이수 여부, 최대 학점 제한 등 사용자가 설정한 조건들이 된다.

본 프로젝트는 CSP 풀이 전략으로 3가지의 기법을 결합한다.

첫째, 백트래킹(Backtracking) 탐색은 과목을 하나씩 배정하다가 제약 조건을 위반하면 즉시 되돌아가 다른 경우를 탐색하는 방식으로, 불필요한 탐색을 조기에 차단한다.

둘째, AC-3 알고리즘(Arc Consistency Algorithm 3)은 탐색을 시작하기 이전에 각 과목 간의 제약 관계를 분석하여 애초에 불가능한 시간 슬롯을 도메인에서 제거한다. 이를 통해서 탐색 공간 자체를 사전에 대폭 축소할 수 있다.

셋째, MRV(Minimum Remaining Values) 휴리스틱은 가능한 시간 슬롯이 가장 적게 남은 과목부터 우선적으로 배정함으로써 충돌을 조기에 감지하고 탐색 효율을 높인다. 이 세 기법의 결합으로 Brute-Force 방식 대비 탐색 공간을 수십만 배 이상 줄일 수 있으며, 수억 가지의 조합도 수초 내에 최적해로 수렴시킬 수 있다.

2.3 챗봇 인터페이스를 결합하는 이유

제약 조건을 입력하는 방식으로는 자연어 챗봇 인터페이스를 제공한다. "금요일 공강이고 아침 수업은 싫어, 알고리즘 수업은 무조건 넣어줘"와 같은 표현을 입력하면 시스템이 이를 파싱하여 CSP 제약 조건으로 자동 변환한다. 챗봇은 첫 대화 시작 시 공강 요일, 제외 교시, 필수 과목 등 필요한 조건을 순서대로 질문하여 사용자가 무엇을 입력해야 할지 자연스럽게 안내한다. 자연어 처리는 LLM API를 활용하여 사용자의 발화에서 요일, 교시, 과목명, 조건 유형을 추출하고 구조화된 제약 객체로 변환하는 방식으로 구현한다. 입력된 조건은 태그 형태로 화면에 표시되어 사용자가 현재 설정된 조건을 한눈에 확인하고 수정할 수 있다.

또한 챗봇은 시간표 생성 이후에도 대화형 피드백 창구로 기능한다. 제약 조건 간의 충돌이 발생했을 때 단순하게 실패를 반환하는 것이 아니라, "금요일 공강을 유지하면 알고리즘 수업을 포함할 수 없습니다. 금요일 공강 조건을 해제하면 몇 개의 후보 시간표가 생성됩니다"와 같이 구체적인 대안을 제시한다. 이를 통해서 사용자는 자신의 우선순위를 재조정하여 최적의 수강 계획을 대화 형식으로 완성할 수 있다.

2.4 고려하는 제약 조건의 범위

본 시스템이 처리하는 제약 조건은 반드시 충족해야 하는 하드 제약과 선호도에 따라 점수화되는 소프트 제약으로 구분된다.

하드 제약은 위반 시에 시간표 자체가 무효가 되는 조건이다. 동일 시간대 과목 중복으로 배정 금지, 학기당 최대·최소 학점 제한 등이 이에 해당한다.

소프트 제약은 사용자의 선호를 반영하여 시간표 품질을 점수화하는 조건들이다. 공강 요일 설정, 아침·야간 수업 제외, 하루 최대 수업 시간 수 제한, 강의실 간 이동거리 최소화, 대면·온라인 수업 비율 조정, 복수 전공 이수 학점 배분 등이 이에 해당된다. 시스템은 소프트 제약 충족도를 점수로 환산하여 여러 후보 시간표를 순위별로 제시한다.

2.5 데이터 수집 전략

강의 데이터는 단국대학교 웹정보 시스템-강의 시간표를 대상으로 Playwright 기반 크롤러를 통해서 수집한다. 수집 항목은 수강 대상 학과, 수강 권장 학년, 이수 구분, 교과목 코드, 과목명, 과목 분반, 학점, 교수명, 날짜, 장소이며 강의 온라인·오프라인 여부는 강의 계획서 상세 페이지를 크롤링 할 예정이다. 데이터를 수집한 후에는, CSP 엔진이 바로 처리할 수 있는 JSON 형식으로 변환하여 저장한다.

3. 기대 효과

본 프로젝트가 완성되면 학생들은 수강 신청을 준비하기 위해 소비하던 많은 시간을 단축할 수 있게 된다. 현재 학생들이 시간표 한 개를 완성하기까지 평균 수 시간을 소비하는 반면, CSP 알고리즘 기반의 자동 생성 방식은 수초 내에 수십 개의 최적 후보 시간표를 동시에 제시할 수 있다. 이는 단순한 편의성 향상을 넘어서, 제한된 시간 안에서 학생이 더 신중하고 전략적인 선택을 내릴 수 있도록 돕는다.

특히 복수 전공생, 편입생, 졸업 요건이 복잡한 고학년 학생처럼 제약 조건이 많은 사용자일수록 효용이 크다. 이들은 두 전공의 필수 과목을 동시에 충족해야 하거나, 잔여 이수 학점과 선수 과목 이수 여부를 일일이 대조해야 하는 등 기존 도구로는 해결하기 어려운 복합적인 조건을 안고 있다. 본 시스템은 이러한 조건들을 한꺼번에 처리하여 실질적인 부담을 줄여준다.

또한 단순하게 시간표를 생성하는 것에 그치지 않고, 조건 충돌 시에 어떤 제약을 완화하면 해결되는지를 사용자에게 피드백을 함으로써 수강 계획에 대한 이해도와 의사 결정 능력을 높이는데 기여한다. 예를 들어 특정 요일 공강과 특정 전공 필수 과목이 동시에 충족되지 않을 경우, 구체적인 대안을 제시한다. 즉, 사용자가 스스로 우선 순위를 조정하며 최적의 수강 계획을 세울 수 있도록 유도한다.

본 시스템은 기존 에브리타임 마법사가 경우의 수 초과 시 실행이 차단되던 한계를 극복하고, 조건이 복잡할수록 더 효과적으로 동작하도록 설계되었다. 나아가 오픈소스로 공개하여 단국대학교를 시작으로 타 대학으로 확장 가능한 범용 구조를 지향함으로써 동일한 문제를 겪는 더 많은 학생들에게 실질적인 도움을 제공하는 것을 목표로 한다.

II. 제공 기능 섹션 구성

1. 기능 목록

1.1 강의 데이터 수집 및 관리

단국대학교 강의 시간표 페이지 대상으로 Playwright 기반 크롤러를 통해 매 학기 강의 데이터를 자동으로 수집한다. 수집 항목은 과목 코드, 과목명, 담당 교수, 강의 시간 및 강의실, 이수 구분, 대면/원격 수업 여부 등이다. 사용자는 챗봇에 "금요일 공강이고 아침 수업은 싫어, 운영체제는 무조건 넣어줘"와 같은 일상적인 표현으로 조건을 입력할 수 있다. 입력된 자연어는 키워드 매칭으로 1차 파싱하고, 모호한 표현은 Gemini

API를 호출하여 구조화된 제약 조건으로 변환한다. 설정된 조건은 태그 형태로 화면에 표시되어 사용자가 한눈에 확인하고 수정할 수 있다.

1.2 자연어 기반 조건 입력 (챗봇)

사용자는 챗봇에 "금요일 공강 만들고 아침 수업은 싫어, 자료구조는 무조건 넣어줘"와 같은 일상적인 표현으로 조건을 입력할 수 있다. 입력된 자연어는 키워드 매칭으로 1차 파싱하고, 모호한 표현은 Gemini API를 호출하여 구조화된 제약 조건으로 변환한다. 설정된 조건은 태그 형태로 화면에 표시되어 사용자가 한눈에 확인하고 수정할 수 있다.

1.3 CSP 기반 시간표 자동 생성

사용자가 입력한 제약 조건을 CSP 엔진에 전달하면 백트래킹 탐색과 AC-3 알고리즘, MRV 휴리스틱을 결합하여 모든 하드 제약을 만족하는 시간표를 자동으로 생성한다. 탐색 전 AC-3 알고리즘으로 불가능한 시간 슬롯을 미리 제거하고, MRV 휴리스틱으로 선택지가 가장 적은 과목부터 우선 배정하여 탐색 공간을 대폭 줄인다. 어떤 조건 입력에도 3초 이내에 결과 또는 부분해를 반환하며, 탐색이 진행되는 동안 조건 분석, 불가능한 시간대 제거, 최적 조합 탐색 등 단계별 진행 상황을 실시간으로 화면에 표시하여 사용자의 체감 대기 시간을 최소화한다.

1.4 후보 시간표 비교 및 선택

CSP 엔진은 하드 제약을 만족하는 해를 하나만 반환하는 것이 아니라 소프트 제약 충족도를 점수화하여 상위 후보 시간표를 복수로 제시한다. 각 후보는 공강 충족 여부, 아침 수업 포함 여부, 하루 최대 수업 교시 수, 건물 간 이동 거리 등의 항목별 점수로 구성된 Breakdown을 함께 표시하여 사용자가 어떤 시간표가 왜 높은 점수를 받았는지 직관적으로 파악할 수 있다. 사용자는 후보 시간표를 좌우로 넘겨보며 비교한 뒤 원하는 시간표를 선택할 수 있다.

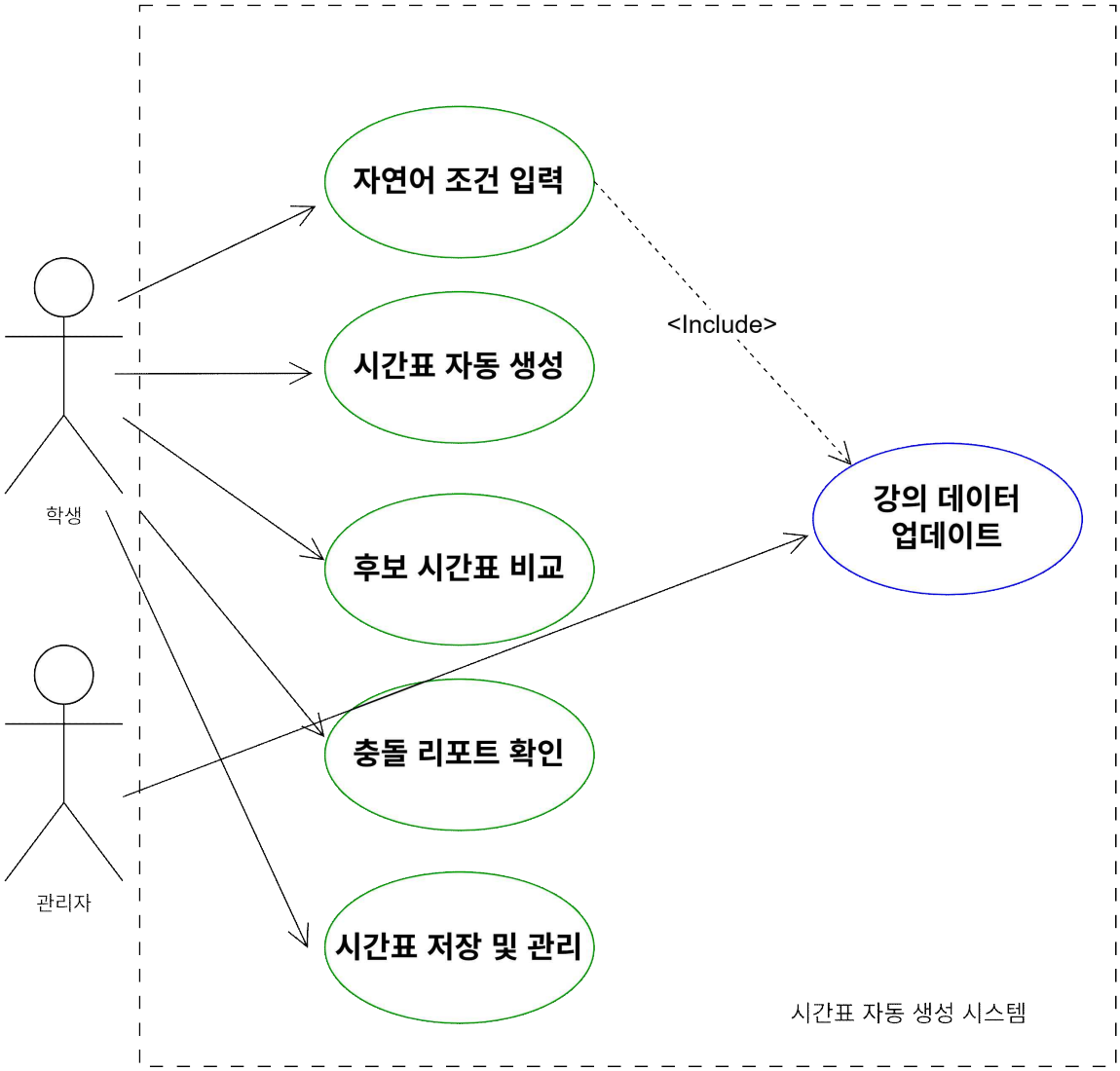
1.5 충돌 리포트 및 조건 완화 제안

사용자가 입력한 제약 조건들이 서로가 모순되어 시간표를 생성할 수 없는 경우, 단순히 실패를 반환하는 것이 아니라 충돌 원인과 구체적 해결 방법을 함께 제시한다. 예를 들어 금요일 공강 조건과 금요일에만 개설된 전공 필수 과목이 동시에 설정된 경우 "금요일 공강 조건을 해제하면 5개의 후보 시간표가 생성됩니다"와 같이 조건 완화 시 생성 가능한 후보 수를 구체적으로 안내한다. 충돌 유형은 동일 시간대 과목 중복, 공강 조건과 필수 과목 간 모순, 학점 초과 등으로 구분하여 표시하며, 사용자는 챗봇 대화를 통해 우선순위를 재조정하며 최적의 수강 계획을 완성할 수 있다.

1.6 시간표 저장 및 관리

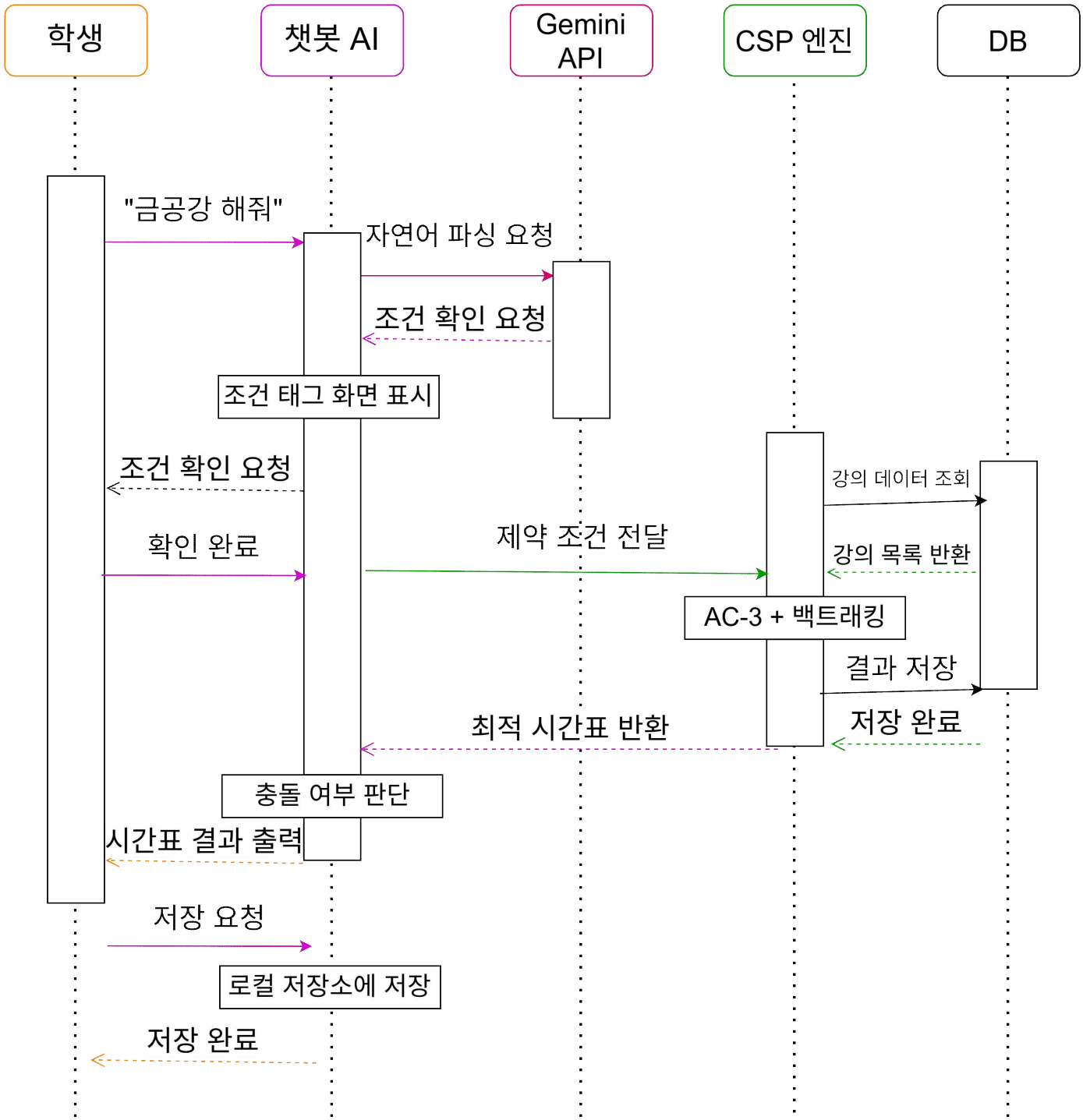
생성된 시간표는 브라우저 로컬 저장소에 저장되어 페이지를 닫은 후에도 다시 불러올 수 있다. 저장된 시간표는 목록 형태로 관리되며 학기별로 구분하여 이전 학기 시간표도 함께 보관할 수 있다. 이를 통해 사용자는 매 학기 시간표를 새로 생성하더라도 이전 학기 기록을 참고하며 수강 계획을 세울 수 있다.

2. 유스케이스 다이어그램



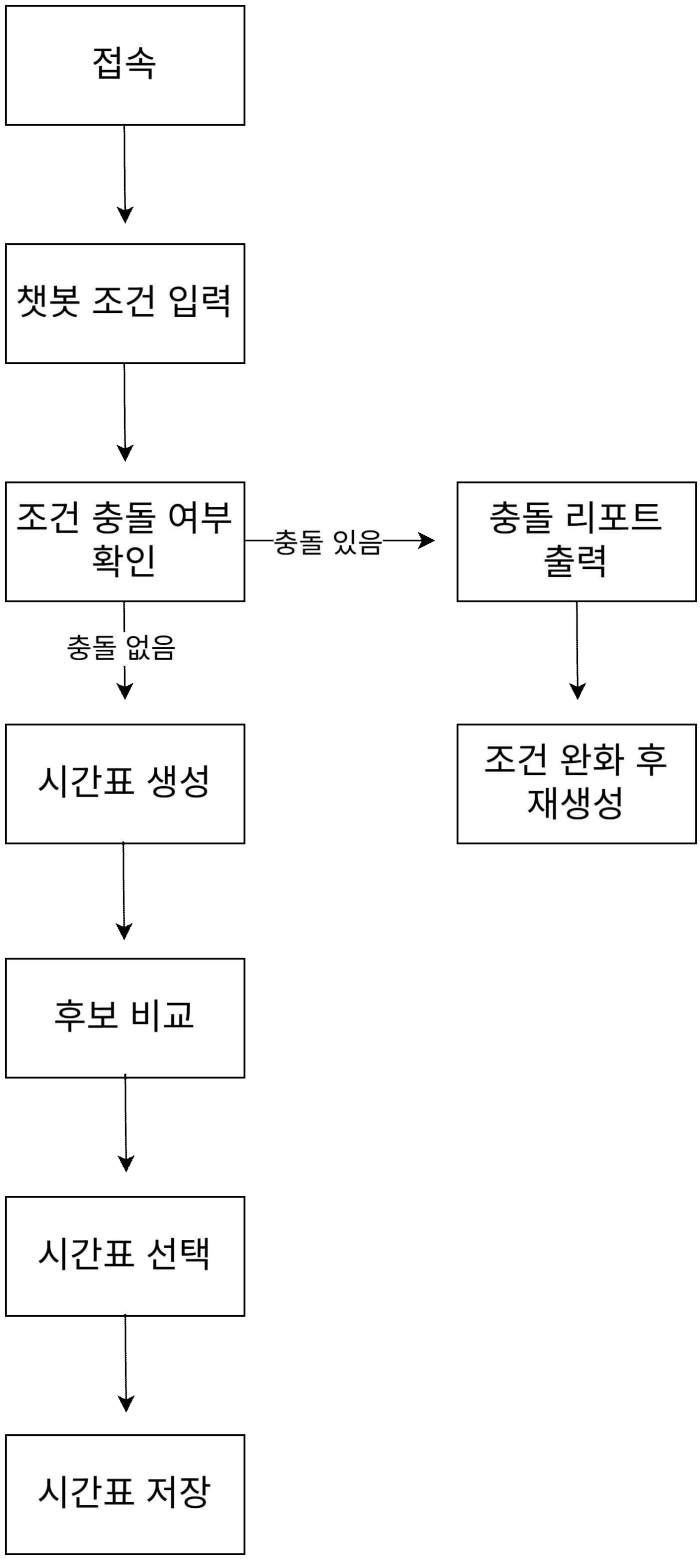
학생은 자연어 조건 입력, 시간표 자동 생성, 후보 시간표 비교, 충돌 리포트 확인, 시간표 저장 및 관리 총 5가지 기능을 사용할 수 있으며, 관리자는 매 학기 강의 데이터 업데이트를 담당한다. 실선은 액터가 해당 기능을 직접 사용하는 관계를 나타내고, 점선<include>는 자연어 조건 입력 기능이 동작하기 위해 강의 데이터 업데이트가 전제되어야 함을 의미한다.

3. 시퀀스 다이어그램



시간표 생성 요청이 들어왔을 때 시스템 내부에서 각 컴포넌트 간에 어떤 순서로 메시지가 오고 가는지를 나타낸 다이어그램이다. 학생이 챗봇 UI에 자연어로 조건을 입력하면 Gemini API가 이를 파싱하여 제약 객체로 변환하고, 챗봇 UI가 조건을 사용자에게 확인받은 뒤 CSP 엔진에 전달한다. CSP 엔진은 DB에서 강의 데이터를 조회한 후 AC-3 알고리즘과 백트래킹으로 최적 시간표를 탐색하여 결과를 반환하며,
최종 시간표는 브라우저 로컬 저장소에 저장된다. 실선 화살표는 요청을, 점선 화살표는 응답을 나타낸다.

4. 플로우 차트



챗봇에 조건을 입력한 후 제약 조건 간 충돌 여부에 따라 흐름이 분기된다. 충돌이 없으면 바로 시간표 생성 및 후보 비교 단계로 넘어가고, 충돌이 있으면 충돌 리포트를 출력하여 사용자가 조건을 완화한 뒤 재생성하는 경로로

이어진다. 최종적으로 원하는 시간표를 선택하고 저장하는 것으로 흐름이 종료된다.

5. 사용자 시나리오

시나리오1 – 일반적인 사용(단일 전공생)
단국대학교 소프트웨어학과 3학년 김단국은 이번 학기 최소 하루 공강을 원하고, 오픈소스SW기초와 운영체제 과목은 반드시 수강해야한다. 아침 수업도 피하고 싶어서 서비스에 접속했다.

① 챗봇에 조건을 입력한다. "최소 하루 공강을 원하고, 운영체제와 오픈소스SW기초는 무조건 넣어줘, 아침 수업은 싫어"
② Gemini API가 자연어를 파싱하여 조건 태그로 변환한다. [금공강] [운영체제 필수] [오픈소스SW기초 필수] [1,2교시 제외]
③ 시스템이 조건을 확인한다. "위 조건으로 시간표를 생성할까요?"
④ 확인 버튼을 누르면 CSP 엔진이 0.3초만에 여러 개의 후보 시간표를 생성한다.
⑤ 점수 순으로 정렬된 후보 중 1위 시간표(92점)을 선택한다.
⑥ 브라우저 로컬 저장소에 저장하여 이후에도 불러올 수 있다.

시나리오2 – 충돌 발생(복수 전공생)
단국대학교 경제학과와 통계데이터사이언스학과를 복수 전공 중인 이단국은 두 전공 필수 과목을 동시에 수강하면서 금공강도 확보하고 싶어한다.

① 챗봇에 조건을 입력한다. "수공강이고 미시경제학1과 수리통계학1 둘 다 넣어줘"
② CSP 엔진이 조건을 분석하던 중 충돌을 감지한다.
③ 충돌 리포트가 출력된다. "수요일 공강을 유지하면 미시경제학1을 포함할 수 없습니다. 수공강 조건을 해제하면 5개 후보 시간표가 생성됩니다."
④ 이단국은 수공강 조건을 해제하기로 결정한다. [금공강] 태그 삭제
⑤ CSP 엔진이 재탐색하여 5개 후보 시간표를 생성한다.
⑥ 후보 중 두 전공 과목이 모두 포함된 시간표를 선택하고 저장한다.

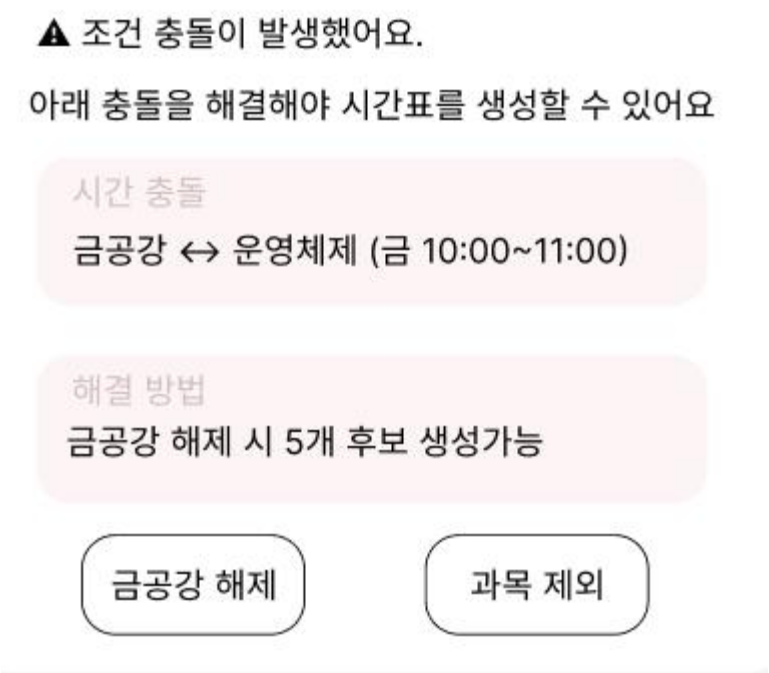
시나리오3 – 이전 학기 시간표 참고(고학년)
졸업을 앞둔 4학년 박단국은 지난 학기 시간표를 참고하면서 이번 학기 시간표를 짜고 싶어한다.

① 서비스에 접속하면 이전 학기에 저장해둔 시간표 목록이 표시된다.
② 저장 목록에서 2025년 2학기 시간표를 불러와 조건을 확인한다. [목공강] [18학점] [전공 위주]
③ 지난 학기와 동일한 조건에서 필수 과목만 변경한다. [캡스톤디자인 필수] 추가
④ CSP 엔진이 조건을 만족하는 후보 시간표를 생성한다.
⑤ 후보 중 원하는 시간표를 선택하고 2026년 1학기로 저장한다.

6. 메뉴 화면 구성

메인 화면

저장된 시간표 목록



충돌 발생 시 나타나는 화면 UI

III. 개발 전략

1. 개발 일정

주차별 계획	상세 계획
1주차(4/3~4/12) - 설계 + 기반 구축	• 요구사항 확정, 데이터 구조 설계 • UI 와이어프레임 제작 • GitHub 세팅, 개발 환경 구축 • 크롤러 개발 착수, 강의 데이터 수집 • Gemini API 연동 기초
시험 기간(4/13~4/24) - 최소 작업	• 개발 사실상 중단 • 여유 있을 때 CSP 알고리즘 설계 문서 작성 검토
3~5주차(4/25~5/14) - 핵심 기능 집중 개발	• CSP 알고리즘 구현 완성 • 챗봇 파싱 모듈, Flask API 서버 • tui-calendar 연동, 시간표 그리드 작업 • Chart.js 시각화, 저장 기능
6주차(5/15~5/21) - 통합	• 프론트와 백엔드 API 연동 • 전체 흐름 테스트 • 충돌 리포트 기능 구현 작업
7주차(5/22~5/28) - 마무리 작업	• 버그 수정, UI 개선 • GitHub 최종 정리 • 발표 자료 제작 • 시연 시나리오 준비
8주차(5/29~6/4) - 발표 준비	• 최종 버그 수정 • 발표 리허설 • 최종 프로젝트 발표

2. 역할 분담

팀원	역할	기술
한상윤	CSP 알고리즘, Flask 백엔드, 크롤러	Python, Flask
박현준	Gemini API 연동, 챗봇 파싱 모듈	Python, JavaScript
양진혁	HTML/CSS 레이아웃, tui-calendar 연동	HTML, CSS, JavaScript
안병규	Chart.js 시각화, 저장 기능, 반응형	JavaScript, Chart.js

3. 필요 기술 스택

영역	기술
프론트앤드	HTML, CSS, JavaScript
시간표 UI	tui-calendar(NHN 오픈소스)
데이터 시각화	Chart.js
챗봇 파싱	키워드 매칭 + Gemini API(Google AI Studio)
CSP 알고리즘	Python, Backtracking + Forward Checking
백엔드 API	Flask
버전 관리	GitHub (main/dev/feature/*)

4. 위험 요인 및 대책

4.1 CSP 탐색 시간 초과

수강 가능한 과목이 많아질수록 조합의 수가 기하급수적으로 늘어난다. 예를 들어, 과목 10개에 분반이 각 3개씩 있으면 탐색해야 할 경우의 수는 $3^{10} = 59{,}049$가지에 달하며, 과목 수가 늘어날수록 이 수치는 폭발적으로 증가할 것이다. 실제 수강신청 상황인 과목 15개 이상, 분반 3~5개 조건에서는 브루트포스 방식으로 수십 초에서 수 시간이 소요될 수 있다. 기존 에브리타임 마법사는 이러한 브루트포스 방식의 한계로 인해 경우의 수가 일정 기준을 초과하면 실행 자체가 차단되는 문제가 있었다.

본 시스템은 AC-3 알고리즘으로 탐색 전에 불가능한 슬롯을 미리 제거하고, MRV 휴리스틱으로 충돌 가능성이 높은 과목부터 우선 탐색하여 같은 조건에서의 에브리타임 마법사보다 더 많은 과목을 처리할 수 있다. 단순히 실행을 차단하는 것이 아니라, 탐색 공간 자체를 줄이는 방식으로 문제를 해결한다. 단순히 실행을 차단하는 것이 아니라 탐색 공간 자체를 줄이는 방식으로 문제를 해결한다. 다만 과목 수가 지나치게 많아지는 극단적인 경우를 대비해 탐색 과목 수 10개 이하 제한과 타임아웃 처리를 안전장치로 추가한다.

또한 탐색 시간이 길어지는 경우에도 사용자가 빈 화면에서 막연히 기다리지 않도록 단계별 진행 상황을 실시간으로 표시한다. 조건 분석, 불가능한 시간대 제거, 최적 조합 탐색 등 각 단계를 순서대로 보여주어 체감 대기 시간을 줄이고 사용자 경험을 개선한다.

4.2 프론트 ↔ 백엔드 통합 지연

팀원 A, B가 백엔드를 개발하는 동안 팀원 C, D는 프론트엔드를 개발한다. 서로 다른 파트를 독립적으로 개발하다가 나중에 합칠 때 API 형식이 맞지 않거나 데이터 구조가 달라서 연동이 안 되는 경우가 생긴다. 특히 개발 경험이 적은 팀원이 있을 경우 이 문제가 더 자주 발생한다.

4월 말에 API 인터페이스를 먼저 확정하고 문서화한다. 백엔드가 완성되기 전까지는 Mock 데이터로 프론트엔드 개발을 진행하여 두 파트가 독립적으로 병렬 개발할 수 있도록 한다.

4.3 Gemini API 무료 한도 초과

Gemini API 무료 티어는 하루 250건으로 제한되어 있다.
사용자가 조건을 입력할 때마다 API를 호출하면 테스트 단계에서 금방 한도를 초과할 수 있다.

모든 입력을 API에 보내지 않고 키워드 매칭으로 1차 처리한다. "금공강", "아침 제외", "1교시" 같이 명확한 키워드는 API 호출 없이 직접 파싱하고, 키워드로 판단하기 어려운 모호한 표현일 때만 Gemini API를 호출한다.
이 방식으로 실제 API 호출 횟수를 전체의 20~30% 수준으로 줄일 수 있다.

4.4 팀원 간 코드 충돌

여러 명이 같은 파일을 동시에 수정하면 Git에서 충돌이 발생한다. 특히 GitHub 사용이 처음인 팀원이 있으면 브랜치 없이 main에 직접 push하거나, pull 없이 작업해서 충돌이 나는 경우가 많다.

브랜치 전략을 명확히 정하고 팀 전체가 이를 따른다.
main 브랜치에는 직접 push하지 않고,
반드시 feature 브랜치에서 작업 후 Pull Request를 통해 merge한다.
매주 오픈소스 수업 마친 후 코드 리뷰를 진행한다.

브랜치 전략
main → 최종 배포본만 merge
dev → 개발 통합 브랜치
feature/csp → 팀원 A 작업
feature/chat → 팀원 B 작업
feature/ui → 팀원 C 작업
feature/chart→ 팀원 D 작업

4.5 처음 팀원 개발 속도 지연

팀원 중 일부는 프로그래밍이 처음이라 HTML/CSS, JavaScript에 익숙하지 않다.
모르는 것이 생겼을 때 혼자 해결하려다 시간을 많이 소비하거나, 개발 환경 세팅부터 막히는 경우가 생길 수 있다.

초기에 일부 팀원들이 개발 환경 세팅을 도와주고, 처음 1~2주는 HTML/CSS 기초부터 시작한다. tui-calendar와 Chart.js는 공식 예제 코드가 잘 되어 있어서 따라하면서 익힐 수 있는 수준이다. 막히는 부분은 혼자 1시간 이상 붙잡지 말고 바로 단톡방에 올려서 팀원에게 물어보도록 한다.

4.6 tui-calendar 커스터마이징 한계

tui-calendar는 NHN에서 만든 오픈소스 캘린더 라이브러리인데,
시간표 형태로 커스터마이징하는 데 제약이 있을 수 있다.
원하는 디자인이나 기능이 라이브러리 자체에서 지원되지 않는 경우가 생길 수 있다.

사전에 FullCalendar를 대체 라이브러리로 검토해둔다.
tui-calendar로 구현이 어렵다고 판단되면 4월 말 이전에 빠르게 교체 결정을 내린다.
교체 결정이 늦어질수록 작업이 낭비되기 때문에 조기에 판단하는 것이 중요하다.

5. 성공 여부 판별 기준

다음 기준을 모두 충족할 경우 프로젝트 성공으로 판정한다.

기준	목표
기능적 완성도	제약 조건 (동일 시간대 과목 중복으로 배정 금지, 학기당 최대·최소 학점 제한, 필수 과목, 공강 시간 배치 등)을 100% 만족하는 시간표 생성
챗봇 인식률	사전 정의된 조건 유형에 대해 90% 이상 정확하게 파싱
성능	어떤 조건 입력에도 10초 이내 결과 또는 부분해 반환
UI 완성도	시간표 그리드, 챗봇 창, 통계 차트 모두 정상 동작
발표 시연	실시간 챗봇 입력 → 시간표 생성 흐름 라이브 시연

IV. 용어 설명 및 참고 문헌

1. 용어 설명

- **CSP (Constraint Satisfaction Problem)** : 변수, 도메인, 제약 조건으로 문제를 정의하고, 모든 제약을 만족하는 변수 값의 조합을 찾는 문제 유형
- **Backtracking** : 탐색 중 제약 위반이 발생하면 이전 단계로 되돌아가 다른 선택을 시도하는 완전 탐색 알고리즘
- **Forward Checking** : 변수에 값을 배정할 때 아직 배정되지 않은 변수의 도메인을 미리 줄여 탐색 공간을 축소하는 기법
- **Hard Constraint** : 반드시 충족되어야 하는 제약 조건 (예: 시간 겹침 없음, 공강 시간)
- **Soft Constraint** : 가능하면 충족하는 것이 좋지만 위반이 허용되는 조건 (예: 하루 최대 과목 수)
- **tui-calendar** : NHN에서 개발한 오픈소스 JavaScript 캘린더/일정 관리 UI 컴포넌트
- **Chart.js** : HTML5 Canvas 기반의 오픈소스 JavaScript 데이터 시각화 라이브러리
- **Brute-Force Enumeration (브루트포스 열거법)** : 가능한 모든 경우의 수를 하나씩 직접 확인하여 해를 찾는 탐색 방법이다. 구현이 단순하다는 장점이 있지만, 탐색 대상이 늘어날수록 계산량이 기하급수적으로 증가하여 현실적인 시간 내에 해를 구하지 못하는 경우가 발생한다.

- AC-3 (Arc Consistency Algorithm 3, 아크 일관성 알고리즘) : CSP에서 탐색을 시작하기 전에 각 변수 간의 제약 관계를 분석하여 애초에 불가능한 값을 도메인에서 미리 제거하는 전처리 알고리즘이다. 예를 들어 A 과목이 월요일 1교시로 배정될 경우 같은 시간대의 B 과목 슬롯을 탐색 전에 제거함으로써 실제 탐색 공간을 대폭 줄인다. 이를 통해 백트래킹 탐색의 효율을 크게 높일 수 있다.

- MRV(Minimum Remaining Values) 휴리스틱 : CSP 탐색 과정에서 아직 값이 배정되지 않은 변수 중 가능한 선택지가 가장 적게 남은 변수부터 우선적으로 탐색하는 전략이다. 선택지가 적은 변수를 먼저 처리하면 충돌을 조기에 발견할 수 있어 불필요한 탐색 경로를 빠르게 제거할 수가 있다.

- 파싱 (Parsing) : 주어진 텍스트나 데이터를 분석하여 의미 있는 구조로 변환하는 과정이다. 본 프로젝트에서는 사용자가 입력한 자연어 문장에서 공강 요일, 제외 교시, 필수 과목 등의 제약 조건을 추출하여 시스템이 처리할 수 있는 형태로 변환하는 작업을 의미한다. 예를 들어 "금공강이고 아침은 싫어"라는 입력을 {"no_class_days": ["금"], "no_class_periods": [1, 2]}와 같은 구조화된 데이터로 변환하는 것이 이에 해당한다.

2. 참고 문헌

- Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.).Pearson. — Chapter 6: Constraint Satisfaction Problems
- NHN tui.calendar GitHub Repository https://github.com/nhn/tui.calendar - 시간표 UI에 사용하는 캘린더 라이브러리
- Microsoft Playwright Documentation. https://playwright.dev/python/docs/intro - 크롤러 구현에 사용하는 라이브러리 공식 문서
- Chart.js. Documentation. https://www.chartjs.org/docs - 데이터 시각화 라이브러리 공식 문서
- Google Gemini API Documentation. https://ai.google.dev/gemini-api/docs -챗봇 파싱에 사용하는 Gemini API 공식 문서
- Pallets Projects. Flask Documentation. https://flask.palletsprojects.com — 백엔드 API 서버 구현에 사용하는 프레임워크 공식 문서

GitHub 링크 : https://github.com/jasonhahn24/dku-timetable